

لو قصدك إنك عايز تروح لل فولدر:



D:\trydocker

في WSL مش هتكتب D/trydocker .

اكتب:



Bash </>

وبعدين:



Bash </>

المفروض:



/mnt/d/trydocker

وال prompt عندك هيظهر:



→ trydocker

لكن مهم

بما إننا بنجهز بيئة Python + Docker + Git، الأفضل عادةً إن مشاريع الشغل تكون داخل WSL:



~/projects/trydocker

وليس:



/mnt/d/trydocker

خصوصًا لو المشروع فيه venv أو node_modules أو ملفات كثيرة، لأن الأداء مع Linux filesystem داخل WSL يكون أفضل.

لو trydocker هو مشروعك الحالي، نقدر ننقله للـ WSL بطريقة صحيحة ونكمل عليه Docker + Python + uv.



حلو جدا فهمني بقا الي حصل من الاول الي الان
علشان لو غيرت الجهاز في المستقبل اقدر اعمل ده تاني

و فهمني انا غيندلوقتي

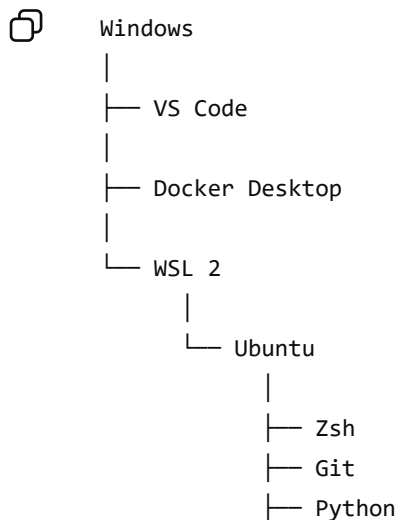
أكيد. خيلنا نرتب كل اللي حصل من أول ما بدأنا لحد دلوقتتي، والأهم إنك تفهم ليه عملناه، بحيث لو
غيرت الجهاز تقدر تعيد كل حاجة من الصفر بنفسك.

1. إحنا بنبني إيه أصلًا؟

أنت عندك Windows، وعايز بيئة Development احترافية لـ:

- Python
- Docker
- Git
- VS Code
- Terminal مريح وذكي

فبنخلي الـ architecture بالشكل ده:



└ uv
└ Projects

الفكرة المهمة:

Windows = الواجهة والبرامج الأساسية

WSL Ubuntu = Linux development environment

2. إيه هو WSL؟

WSL = Windows Subsystem for Linux

يعني Windows بيسمح لك تشغل Linux environment داخله.

بدل ما تعمل:



Windows

فقط، بقى عندك:



Windows

|

└ WSL

|

└ Ubuntu

وده مهم جدًا لأن أغلب الـ servers والـ Docker environments في العالم الحقيقي شغالة Linux.

فأنت بتطور على بيئة قريبة من Production.

3. ليه Ubuntu؟

WSL ممكن يشغل توزيعات مختلفة.

إحنا استخدمنا Ubuntu.

فبقى عندك:



Windows

|

└ Ubuntu

وال Ubuntu ده عنده filesystem خاص بيه:



/home/elbostan

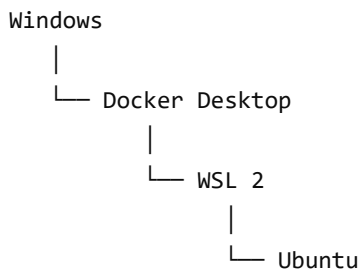
وده هو المكان اللي هنحط فيه مشاريعنا.

4. طيب إيه اللي حصل مع Docker؟

أنت عندك Docker Desktop على Windows.

Docker Desktop بيستخدم WSL 2.

يعني تقريبًا:



وبالتالي من داخل Ubuntu تقدر تعمل:



Bash </>

بدون ما تحتاج تثبيت Docker Engine داخل Ubuntu نفسه.

وده مهم:

لا تعمل:



Bash </>

طالما أنت تستخدم Docker Desktop + WSL integration.

لأن Docker Desktop هو اللي بيدير Docker Engine.

5. طيب إيه موضوع VS Code؟

VS Code نفسه موجود على Windows.

لكن لما تدخل مشروع من WSL:



Bash </>

VS Code يعمل حاجة ذكية.

الواجهة:



Windows



VS Code

لكن ال development environment:



WSL



Ubuntu

فتلاقى تحت في VS Code:



WSL: Ubuntu

معناه:

VS Code حاليًا متصل بالـ Ubuntu بتاعك.

6. طيب ليه قلنا المشاريع داخل WSL؟

عندك طريقتين.

Windows filesystem

مثلًا:



D:\trydocker

ومن WSL:



/mnt/d/trydocker

لأن:



D: /

في Windows تصبج:



/mnt/d/

Linux filesystem

داخل WSL:



/home/elbostan/projects

أو ببساطة:



Bash </>

وده المكان اللي أنصحك تحط فيه مشاريع Linux/Python/Docker.

مثلاً:



/home/elbostan/projects/

├─ trydocker

├─ my-api

└─ bot

7. إيه قصة ال hash الغريب؟

دي كانت أهم حاجة حصلت معنا 😄

أنت كنت واقف هنا:



/mnt/wsl/docker-desktop-bind-mounts/Ubuntu/d4b7c48a53c...

وده مش project folder بتاعك.

ده مكان داخلي بيستخدمه Docker Desktop في WSL.

عشان كده ال prompt كان:



→ d4b7c48a53c29017c59d1ef09e7c49c311072e8ac1e892879c2ad3606bb185de

إحنا افكرنا في البداية إن ال prompt نفسه فيه مشكلة.

لكن الحقيقة:



Bash </>

طلع:



d4b7c48a53c...

يعني ببساطة:

أنت كنت واقف في فولدر اسمه فعلاً ال hash ده.

8. طيب ال Prompt عملناه إزاي؟

في البداية Oh My Zsh كان بيعرض:



→ folder git:(master) X

وكان عندك:



git:(master)

وعلامه:



X

إحنا مش عايزين كل ده.

أنت قلت:

عايز اسم الفولدر الحالي بس.

فخليناه:



Bash </>

الجزء المهم:



```
%1~
```

يعني:

اعرض آخر directory فقط.

فلو:



```
/home/elbostan/projects/trydocker
```

هيظهر:



```
→ trydocker
```

ولو:



```
/home/elbostan/projects
```

هيظهر:



```
→ projects
```

9. وإيه هو Zsh؟

ال shell الافتراضي غالبًا يكون:



```
bash
```

إحنا ثبتنا:



```
zsh
```

Zsh = shell زي Bash، لكن فيه features وإمكانيات أكثر.

وبعدين ثبتنا:



```
Oh My Zsh
```

Oh My Zsh مش Shell جديد.

يعني:

```

Zsh
↑
Oh My Zsh
↑
Plugins / Theme / Config
```

10. إيه هو . zshrc ؟

ده أهم ملف لازم تحفظ اسمه.

```
~/.zshrc
```

ده configuration file الخاص بـ Zsh.

كل مرة تفتح Terminal جديد، Zsh يقرأ الملف ده.

إحنا حطينا فيه مثلاً:

```
export ZSH="$HOME/.oh-my-zsh"

ZSH_THEME=""

plugins=(git zsh-autosuggestions zsh-syntax-highlighting)

source $ZSH/oh-my-zsh.sh

. "$HOME/.local/bin/env"

PROMPT='%F{cyan}→ %F{white}%1~%f '
```

خلينا نفهم كل واحد.

11. export ZSH



Bash </>

معناه:

Oh My Zsh موجود هنا.

يعني:



```
/home/elbostan/.oh-my-zsh
```

وده ظهر لما عملنا:



Bash </>

وطلع:



```
/home/elbostan/.oh-my-zsh
```

12. ZSH_THEME

إحنا عملنا:



Bash </>

ليه؟

لأننا مش عايزين Theme جاهز يغير ال prompt بتاعنا.

كان قبل كده:



Bash </>

وده كان بيضيف معلومات Git وغيره.

إحنا قلنا:

لا، أنا هتحكم في ال prompt بنفسي.

Plugins .13

عندنا:



Bash </>

يعني فعلنا 3 plugins:

Git



git

يساعد Zsh/Oh My Zsh في تكامل Git.

Autosuggestions



zsh-autosuggestions

يجيب suggestions من ال command history.

Syntax highlighting



zsh-syntax-highlighting

يعمل highlighting للأوامر.

14. ليه echo \$plugins في الأول مظهرش حاجة؟

لأن Oh My Zsh ماكانش متحمل من .zshrc.

بعد ما ضبطناه، ظهر:



git zsh-autosuggestions zsh-syntax-highlighting

وكمان:



Bash </>

بقى:



/home/elbostan/.oh-my-zsh

فده أكد لنا إن configuration اشتغل.

15. طيب ال Autosuggestions بتعمل إيه؟

دي نقطة مهمة.

هي مش AI autocomplete.

مثلاً لو كنت استخدمت:



Bash </>

قبل كده، وكتبت:



Bash </>

ممکن يقترح:



docker compose up

وتضغط:



→

لقبول الاقتراح.

يعني:



History



zsh-autosuggestions



Suggestion

مش:



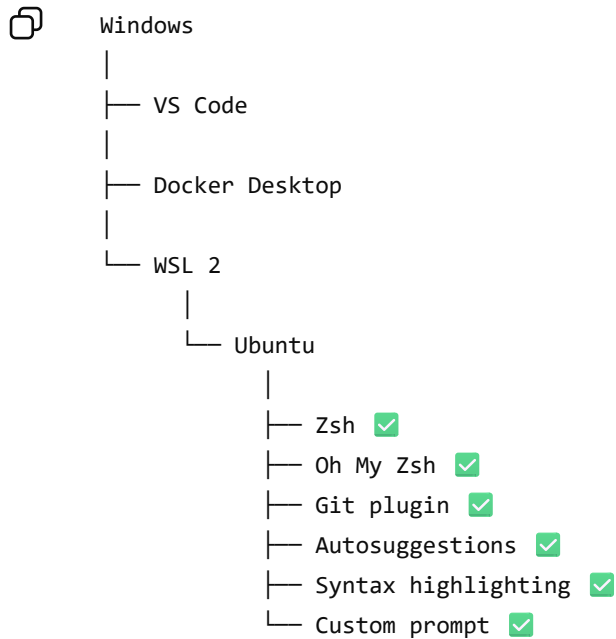
AI



وده سبب إنك ما كنتش شايف suggestions في البداية.

16. إيه وضعك دلوقتي؟

أنت حاليًا وصلت إلى مرحلة ممتازة:



وال prompt عندك أصبح بالشكل اللي طلبته:

→ folder-name

17. وإحنا واقفين فين بالضبط الآن؟

أنت قلت:

D/trydocker

والصح في WSL لو المشروع موجود على D:

Bash </>

فأنت الآن غالبًا تريد التعامل مع:



D:\trydocker

ك project.

لكن لسه ما خالصناش ال Python/Docker/Git setup للمشروع نفسه.

18. أنا أنصحك تعمل إيه الآن؟

بما إن هدفنا النهائي:



```
Python
+
uv
+
Docker
+
Docker Compose
+
Git
+
VS Code
```

فأنا شايف إننا دلوقتي ما نركبش حاجات إضافية عشوائيًا.

نعمل المشروع trydocker صح.

الخطوة التالية تكون:



```
trydocker
|
├─ Git
├─ Python
├─ uv
├─ .venv
├─ pyproject.toml
├─ uv.lock
├─ Dockerfile
├─ compose.yaml
├─ .dockerignore
└─ .gitignore
```

ونفهم ليه كل ملف موجود وإمتى نستخدمه.

ولو غيرت الجهاز مستقبلاً؟

احفظ الخريطة دي:

Windows ①

ثبت:



WSL 2
Ubuntu
VS Code
Docker Desktop

Ubuntu ② داخل

ثبت:



Git
Zsh
Oh My Zsh
zsh-autosuggestions
zsh-syntax-highlighting
uv

VS Code ③

ثبت:



WSL
Python
Pylance
Ruff
Docker
GitLens

المشاريع ④

خليها:



~/projects/

Docker ⑤



Docker Desktop
↓
WSL Integration
↓
Ubuntu

```
cd ~/projects/my-project
```



Bash </>

وبكده عندك نفس بيئة الشغل تقريبًا على الجهاز الجديد.

والأهم: أنت دلوقتى فاهم الفرق بين:



Windows path

D:\trydocker

و:



WSL path

/mnt/d/trydocker

و:



WSL Linux path

/home/elbostan/projects/trydocker

ودي نقطة أساسية جدًا في شغل WSL + Docker.

لو هنكمل من هنا، أنا أنصح أن الخطوة التالية تكون مشروع trydocker نفسه ونبنيه من الصفر بـ uv
Git + Compose + Docker + Python ، ونفهم كل ملف عمليًا بدل ما نحفظ أوامر.



عايز حاجه مناسبة و حلوه غير ال سهم

آه طبعًا 😊 السهم → بسيط، لكن فيه أشكال أنضف وأشيك لل terminal.

بما إنك عايز اسم الفولدر فقط، أنا أرشح واحد من دول:

1. شكل بسيط واحترافي ★



~/trydocker >

أو بما إنك عايز اسم الفولدر فقط:



trydocker >

2. شكل Developer



trydocker »

3. شكل Minimal



trydocker \$

وده كلاسيكي جدًا:



trydocker \$

4. شكل لطيف



trydocker >

5. بدون رموز تقريبًا



[trydocker]

أنا شخصيًا أرشح لك



trydocker >

لأنه واضح جدًا، ويفصل بين اسم المكان ومكان كتابة الأمر.

عشان تعمله، في ~/zshrc غيّر:



Bash </>

إلى:



Bash </>



Bash </>

ثم:



Bash </>

هيبقى:



trydocker >

ولو دخلت:



Bash </>

cd ~/projects/my-api

هيبقى:



my-api >

ولو عايز شكل أكثر احترافية من كده، ممكن نعمله مثل Terminal بتاع developers المحترفين، مثل:



~/projects/trydocker | >

أو:



trydocker | \$

من غير Git ولا Python ولا Docker معلومات زيادة.



ChatGPT can make mistakes. Check important info.